
HARNESS ENGINEERING: A GOVERNANCE FRAMEWORK FOR AI-DRIVEN SOFTWARE ENGINEERING

A PREPRINT

Jiun Kim

Hanyang University, Seoul, Republic of Korea

March 23, 2026

ABSTRACT

Harness engineering—the practice of embedding machine-enforceable architectural constraints in a development toolchain—has emerged independently across organizations navigating AI-driven software engineering, but lacks a shared analytical vocabulary or formal definition. This paper offers an initial conceptual framing. We define a **harness** as a governance system for the structural consistency of code artifacts, and organize harness engineering along three dimensions: (1) **Context**—the declarative and procedural knowledge that informs the agent; (2) **Constraint**—the rules that govern agent output, spanning guidance before code is produced and enforcement after; and (3) **Convergence**—the iterative process by which constraints are evaluated, gaps identified, and rules refined until the harness reaches structural idempotence: a state where re-application produces no further structural change. We illustrate that this tripartite structure recurs across independent practitioners—including OpenAI, HashiCorp, Toss, and Thoughtworks—who converged on similar practices without shared vocabulary. The framework offers a shared analytical vocabulary for an emerging governance practice that shifts attention from constraining *who writes code* to constraining *what code structures are permissible*.

Keywords harness engineering · AI-driven software engineering · software governance · structural consistency · structural idempotence

1 Introduction

1.1 The Emergence of Harness Engineering

In February 2026, OpenAI published “Harness Engineering: Leveraging Codex in an Agent-First World” [Lopopolo, 2026], describing how a team of three engineers built a million-line product with zero manually-written code. The post introduced the term “harness engineering” to describe a practice where the engineer’s role shifts from writing code to “designing environments, specifying intent, and building feedback loops that allow Codex agents to do reliable work.” Concurrently, practitioners such as Shoemaker [2026] advocated abandoning line-by-line code review in favor of spec-driven, harness-first verification, while industry engineers at Toss [Kim, 2026a] proposed organizational harness systems through plugin marketplaces that standardize AI workflow across teams.

These accounts share a common insight: **in a world where code is increasingly generated by AI agents, the constraints around the code matter more than the code itself**. Yet all existing treatments are informal—blog posts, Twitter threads, conference talks—lacking the theoretical precision needed to reason about harness systems rigorously, compare implementations, or predict their properties.

1.2 Precursors: Governance Has Tracked Production

The need for deterministic structural governance is not new; what is new is the urgency created by probabilistic code generation. Prior practices established the principles that harness engineering draws upon. Ruby on Rails popular-

ized *convention over configuration* [Hansson, 2004], reducing structural decisions by embedding governance into the framework’s generators. Schema-driven tools such as Protocol Buffers and Prisma made declarative specifications the single source of code structure, eliminating entire categories of structural variance. Most directly, Infrastructure as Code tools—particularly Terraform, created by Hashimoto [2026]—established *idempotent state convergence* as an engineering norm: regardless of how many times a configuration is applied, the infrastructure converges to the same declared state [Hummer et al., 2013, Morris, 2016]. Static analysis tools evolved from bug detection [Johnson, 1978] toward architectural enforcement [Beller et al., 2016], though their use for project-specific structural governance remained underexplored.

Each of these practices addressed structural variance within its respective production paradigm. What distinguishes the current moment is that LLM-driven code generation has made production itself *probabilistic*: the same prompt can yield structurally different outputs across runs, models, and agents. The governance mechanisms inherited from the deterministic era—documentation, conventions, instruction files—were not designed for this regime. Harness engineering, as we articulate it, is a response to this gap.

1.3 Architectural Entropy in AI-Driven Software Engineering

Software engineering has always contended with variance. Different developers interpret the same requirements differently, structure code differently, and name things differently. Code review, style guides, and pair programming evolved as social mechanisms to manage this variance.

The emergence of AI-driven software engineering has fundamentally altered this equation. LLMs can generate hundreds of lines of syntactically valid, functionally correct code in seconds. The term “vibe coding”—coined by Andrej Karpathy in 2025 [Karpathy, 2025]—describes a development practice where the programmer specifies intent in natural language and the AI produces implementation. When a codebase receives contributions from Human Developer A, Human Developer B, AI Model X, and AI Model Y, each agent brings its own implicit assumptions about file organization, naming conventions, dependency direction, and architectural layering. The result is **architectural entropy**—a gradual erosion of structural coherence.

OpenAI’s team observed this directly: “Codex replicates patterns that already exist in the repository—even uneven or suboptimal ones. Over time, this inevitably leads to drift” [Lopopolo, 2026]. Their response—encoding “golden principles” and running recurring cleanup agents—is an operational description of what we articulate as *convergent constraint enforcement*.

1.4 The Limits of Prompt-Based Governance

Current mitigation strategies for AI code variance are predominantly **probabilistic**:

- **Prompt engineering** (e.g., CLAUDE.md, .cursorrules, AGENTS.md): Increases the probability of compliance but cannot guarantee it.
- **Manual code review**: Human review of AI-generated output, increasingly impractical as AI throughput exceeds human attention capacity.
- **Fine-tuning**: Training models on project-specific corpora, requiring significant investment with diminishing returns across model updates.

OpenAI themselves discovered this limitation: “We tried the ‘one big AGENTS.md’ approach. It failed in predictable ways... Too much guidance becomes *non-guidance*” [Lopopolo, 2026]. The Toss Tech blog articulates the same gap: “Even with the same model and IDE, the variance in output is extreme” [Kim, 2026a].

This paper provides a **complementary perspective**: a governance framework for architectural constraints that aim to produce consistent structural outcomes regardless of authoring agent.

1.5 Contribution

We make three contributions:

1. **Conceptual framework** organizing harness engineering along three dimensions—Context, Constraint, and Convergence—with structural idempotence as the convergence criterion (Section 3).
2. **Convergent evidence** from independent industry practitioners demonstrating the three dimensions in practice (Section 4).

3. **Governance analysis** positioning harness engineering within a pre-generation to post-generation enforcement spectrum (Section 5).

2 Related Work

2.1 Industry Discourse on Harness Engineering

The term “harness engineering” was popularized by OpenAI’s February 2026 blog post [Lopopolo, 2026]. Around the same period, several practitioners independently described similar practices under different names [Zakas, 2026, Shoemaker, 2026, Kim, 2026a]. All accounts share a common insight—that the engineer’s output is shifting from code to constraints—but none provide formal definitions or analytical frameworks. We analyze this convergent emergence as first-class evidence in Section 4.

2.2 Software Architecture Erosion and Technical Debt

Perry and Wolf [1992] introduced the foundational model of software architecture and identified *architecture erosion*—the gradual degradation of architectural integrity as implementation decisions violate intended constraints. de Silva and Balasubramaniam [2012] survey the erosion control literature, categorizing approaches into process-oriented (reviews, documentation), architecture-oriented (formal ADLs, reflexion models), and tool-oriented (conformance checkers). Our work falls in the tool-oriented category but operates at a finer granularity than traditional approaches: file naming, export patterns, and function ordering rather than module-level dependencies.

Baldwin and Clark [2000] formalized *design rules*—constraints that decouple modules and enable independent evolution. Their insight that “a few powerful design rules can govern the behavior of many independent modules” resonates directly with harness engineering: a small set of rules can govern structural decisions across an entire codebase. The key difference is that Baldwin and Clark’s design rules are architectural decisions made by system designers, whereas harness rules are machine-enforced constraints that bind all authoring agents equally.

Cunningham [1992] introduced the *technical debt* metaphor to describe the cost of expedient but structurally suboptimal decisions. Harness engineering can be understood as a preventive mechanism against a specific class of technical debt: structural inconsistency introduced by heterogeneous authoring agents. Where technical debt is typically repaid through refactoring, harness rules prevent the debt from accruing in the first place.

2.3 Static Analysis and Architecture Conformance

Static analysis predates modern software engineering. Lint [Johnson, 1978] analyzed C source code for suspicious constructs. Modern linters (ESLint, Prettier, RuboCop, Clippy) have expanded from bug detection to style enforcement, but their use for *architectural* enforcement remains underexplored in the literature. Beller et al. [2016] conducted a large-scale evaluation of static analysis tool adoption in 122 open-source Java projects, finding that while tools like Checkstyle and PMD are widely configured, their rules rarely enforce architectural constraints beyond basic style.

Architecture conformance tools such as ArchUnit [Gafert, 2019] (Java), deptrac (PHP), and Dependency Cruiser (JavaScript) verify dependency rules but operate at the module or package level. Knodel and Popescu [2007] compared six static conformance checking approaches (reflexion models, relation conformance rules, dependency profiles, among others), all of which target inter-module dependencies. None address the file-naming, export-pattern, or function-ordering level that characterizes day-to-day structural decisions—precisely the level at which harness engineering operates.

Allamanis et al. [2014] proposed a machine learning approach to infer coding conventions from existing codebases, demonstrating that conventions are learnable patterns. This complements our approach: where Allamanis et al. learn conventions *descriptively*, harness engineering enforces them *prescriptively* through deterministic rules.

2.4 Convention over Configuration

Ruby on Rails popularized “convention over configuration” (CoC) [Hansson, 2004], reducing developer decisions by establishing defaults. CoC is effective but framework-specific: it governs routing, ORM mapping, and asset paths within the Rails ecosystem. It does not generalize to arbitrary architectural patterns. Harness engineering extends the CoC philosophy beyond frameworks: custom lint rules encode project-specific conventions as enforceable constraints, applicable to any architecture.

2.5 Idempotency, Confluence, and Convergence in Software Systems

Idempotency is well-studied in distributed systems (HTTP methods [Fielding, 2000]), infrastructure as code (Terraform, Ansible [Morris, 2016]), and database migrations. Hummer et al. [2013] formalized idempotency testing for IaC, demonstrating that configuration scripts should produce identical system states regardless of how many times they are applied. While IaC idempotency concerns *infrastructure state* convergence rather than *code structure* convergence, the underlying principle—that repeated or reordered application of declarative rules should yield the same outcome—is analogous to the confluence property we define for harness engineering. Confluence in the formal sense—the property that different reduction paths yield the same normal form—originates in term rewriting theory [Church and Rosser, 1936] and is widely applied in compiler optimization and programming language semantics. We draw on this latter concept to characterize architectural constraint enforcement, while acknowledging the IaC tradition as an independent exploration of similar convergence properties at a different abstraction level.

3 Harness Engineering: A Conceptual Framework

We organize harness engineering along three dimensions: **Context**, **Constraint**, and **Convergence**. This tripartite structure reflects the fundamental control loop—observe state, apply control, iterate on feedback—established in cybernetics and autonomic computing. Rather than claiming formal proofs or universal guarantees, we articulate these as analytical dimensions grounded in practitioner observation.

3.1 Key Concepts and Definitions

An **authoring agent** is any entity that produces source code: a human developer, an AI coding assistant, or an automated pipeline. A **harness** is a governance system for the structural consistency of code artifacts, composed of machine-enforceable rules coordinated across development tools (linters, type checkers, formatters, CI gates). The **structural signature** of a code artifact is the tuple of its architecturally relevant properties—file path, naming, directory placement, exports, imports, function ordering—explicitly excluding implementation logic.

3.2 Dimension 1: Context

Context is the knowledge that informs the authoring agent. We distinguish two layers:

3.2.1 Declarative Knowledge

Facts, states, and structures that the agent can reference: documentation (docs/), API schemas, codebase structure, database models, and dependency graphs. Declarative knowledge answers *what exists*. Without it, the agent generates code that is structurally plausible but factually wrong—e.g., importing a module that does not exist or using a deprecated API.

3.2.2 Procedural Knowledge

Principles, decision methods, and architectural patterns that guide judgment: naming conventions, layering rules, dependency direction policies, and pattern selection criteria. Procedural knowledge answers *how to decide*. It operates even in the absence of specific declarative knowledge—e.g., “when in doubt, prefer composition over inheritance” applies regardless of the particular codebase.

The Context gap. Current harness engineering discourse conflates these two layers under labels like “context engineering” or “knowledge base.” Distinguishing them is analytically useful because they fail differently: missing declarative knowledge produces *factual errors* (wrong imports, nonexistent APIs), while missing procedural knowledge produces *judgment errors* (correct code in the wrong pattern).

Distinguishing Context from Constraint. Context and Constraint may appear to overlap, especially since both can reside in the same medium (e.g., an AGENTS.md file). The distinction lies not in the medium but in the speech act: Context is *descriptive*—it states what exists, how the system is structured, and what patterns are in use (e.g., “our API uses REST with JSON responses”). Constraint is *prescriptive*—it states what must or must not be done (e.g., “always use kebab-case for file names”). A single instruction file may contain both descriptive context and prescriptive constraints; the framework distinguishes them by function, not by location. Within Constraint, the further distinction between Guidance and Enforcement reflects whether compliance can be mechanically verified: Guidance is prescriptive but unverifiable (the agent may or may not comply), while Enforcement is prescriptive and verifiable

(the toolchain produces a binary pass/fail judgment). The same architectural principle may appear in both dimensions: as a procedural guideline in Context, and as an enforceable rule in Constraint. This is not redundancy but *layered governance*—Context informs the agent’s intent, while Constraint verifies the agent’s output.

3.3 Dimension 2: Constraint

Constraint is the set of rules that govern agent output. We distinguish two layers by their temporal relationship to the authoring event: Guidance (pre-authoring) and Enforcement (post-authoring).

3.3.1 Guidance

Rules that shape the agent’s behavior *before* code is produced: instruction files (AGENTS.md, CLAUDE.md, .cursorrules), system prompts, few-shot examples, and structured task decomposition. Guidance operates probabilistically—it increases the likelihood of compliant output but cannot guarantee it. As OpenAI discovered, “we tried the ‘one big AGENTS.md’ approach. It failed in predictable ways. . . Too much guidance becomes *non-guidance*” [Lopopolo, 2026].

3.3.2 Enforcement

Rules that evaluate and transform code *after* it is produced: linters, type checkers, formatters, architectural conformance tools, CI gates, and autofix transformations. Enforcement operates deterministically—it either passes or rejects code, regardless of which agent produced it.

The enforcement spectrum. Guidance and Enforcement span a spectrum of enforcement strength, shown in Table 1.

Table 1: The enforcement spectrum from Guidance to Enforcement.

Mechanism	Example	Strength	Layer
Documentation	“Please use kebab-case”	None	Guidance
Convention	Team agreement	Social pressure	Guidance
Instruction file	AGENTS.md / CLAUDE.md	LLM-dependent	Guidance
Lint warning	Lint <code>warn</code>	Visible signal	Enforcement
Lint error	Lint <code>error</code>	Blocking gate	Enforcement
Autofix	Lint/formatter autofix	Automatic transformation	Enforcement
Type system	TypeScript compiler	Compile-time rejection	Enforcement

The critical boundary is the mid-table divide. Pre-generation control dominates current harness engineering discourse; post-generation control provides the deterministic guarantees that make structural convergence possible.

Choice-space reduction as the cumulative effect. Together, pre- and post-generation controls progressively reduce the space of permissible structural decisions through three mechanisms of increasing strength: (1) *choice elimination*—mandating a single option (e.g., kebab-case file names), (2) *choice channeling*—constraining to a curated vocabulary (e.g., *Model, *Repository, *Adapter suffixes), and (3) *choice canonicalization*—ensuring that whatever choice is made, autofix converges to an identical structural outcome (e.g., function ordering rules).

Agent-independence as a corollary. When post-generation constraints are sufficiently comprehensive, the authoring agent’s identity becomes irrelevant to structural outcome. Whether Developer A, Developer B, or Claude Code implements a feature, the harness enforces the same structural signature. Code quality (at the structural level) becomes a property of the harness, not of the author.

3.4 Dimension 3: Convergence (Evaluation / Refinement)

Convergence is the iterative process by which a harness evolves toward completeness. Harness development proceeds in stages $H_0 \subset H_1 \subset \dots \subset H_n$, where each H_i addresses patterns that become visible only after H_{i-1} is applied. Four factors make upfront complete specification impractical:

1. **Domain co-evolution.** Constraints can only be written for domains that exist. Agent-specific rules cannot be articulated before the agent subsystem is designed.

2. **Constraint interaction.** Rules interact in non-obvious ways. Each constraint layer reveals interaction effects that demand adjustment.
3. **Pattern emergence.** Architectural anti-patterns are often invisible until a codebase reaches sufficient size.
4. **Economic infeasibility.** Writing all constraints upfront would require predicting every future architectural pattern—a task whose cost grows with codebase complexity.

Structural idempotence as the convergence criterion. The convergence process requires a termination condition: *how do we know the harness is sufficiently complete?* We adopt the criterion of **structural idempotence** [Kim, 2026b]: a harness has converged when re-applying its rules produces no further structural change. Formally, let P denote the composite transformation of all harness rules applied to a codebase state σ , and let O be a structural observation. The harness has converged when $O(P^2(\sigma)) = O(P(\sigma))$.

A key result from Kim [2026b] strengthens this criterion: for *any* well-defined transformation $P: \Sigma \rightarrow \Sigma$, there exists an observation O under which structural idempotence holds. However, this existence claim is only useful when O is non-trivial—a trivial observation that collapses all states into one would satisfy the condition vacuously. In our framework, O is instantiated as the **structural signature** defined in Section 3: the tuple of file path, naming, directory placement, exports, imports, and function ordering. This observation is neither trivial (it distinguishes structurally different artifacts) nor the identity (it deliberately excludes implementation logic). The question is therefore not *whether* a harness can converge, but *whether the chosen structural signature remains stable under re-application*. Furthermore, once $O(P^2(\sigma)) = O(P(\sigma))$ holds, it holds for all subsequent applications: $O(P^k(\sigma)) = O(P(\sigma))$ for all $k \geq 1$. This means convergence can be tested with a single re-application—no infinite iteration is required.

This criterion is operationally testable: run the enforcement pipeline (linters, formatters, type checks) twice; if the second pass produces no changes, the harness has converged for that codebase state. When $O(P^2(\sigma)) \neq O(P(\sigma))$, a gap exists—rules are incomplete or conflicting—and the corrective direction is to refine the rules P , not to change the observation O .

Path-independent convergence. A well-designed harness exhibits path-independent convergence: different orders of rule introduction lead to the same structural outcome. This property, analogous to confluence in term rewriting [Church and Rosser, 1936], makes incremental development safe. Many lint rules are declarative predicates rather than sequential transformations, which makes path independence achievable in practice, though rule interaction and autofix conflicts mean it must be treated as an engineering achievement rather than an automatic guarantee.

3.5 Interdependencies Among the Three Dimensions

The three dimensions form a reinforcing system, summarized in Table 2:

Table 2: The three dimensions of harness engineering and their roles.

Dimension	Question	Role	If absent
Context	What does the agent know?	Informs	Factual & judgment errors
Constraint	What is the agent allowed to produce?	Governs	Unbounded structural variance
Convergence	Has the output structurally converged? ($O(P^2) = O(P)$)	Evaluates	No termination criterion

Context without Constraint provides knowledge but no enforcement. Constraint without Context produces rules disconnected from architectural intent. Both without Convergence yield a static harness that never adapts to the evolving codebase. The three dimensions together constitute a governance loop: Context informs the agent, Constraint governs its output, and Convergence evaluates results and feeds gaps back into Context and Constraint for the next iteration.

4 Harness Engineering in Practice: Convergent Emergence

The conceptual framework of Section 3 was not derived in isolation. To ground the framework in practice, we surveyed practitioner accounts published through March 2026 that describe concrete governance practices for AI-generated code. We selected six accounts that (a) describe specific tools, workflows, or organizational mechanisms rather than general commentary, and (b) were authored with minimal mutual influence—while some sources reference the term “harness engineering” from OpenAI, their described practices were developed independently. The resulting set spans individual engineers, startups, and large organizations across different toolchains and scales. We argue that this **convergent emergence** suggests that the underlying phenomenon is substantive and that a shared vocabulary is needed.

4.1 Independent Discovery Across Scales

4.1.1 OpenAI: Entropy Management at Scale

OpenAI’s “Harness Engineering” post [Lopopolo, 2026] describes a team of three engineers producing a million-line codebase with zero manually-written code. Their central discovery was that AI agents replicate existing patterns—including suboptimal ones—leading to progressive architectural drift. Their response maps to all three dimensions:

- **Context.** A curated docs/ knowledge base, evolved from an initial “one big AGENTS.md” that “failed in predictable ways,” provides declarative and procedural knowledge to agents.
- **Constraint.** Custom linters enforce “golden principles” (dependency direction, naming conventions), producing consistent structural outcomes regardless of which Codex agent executes a task.
- **Convergence.** Recurring cleanup agents perform “entropy management,” iteratively identifying and correcting structural drift between enforcement points.

4.1.2 Toss: Composable Constraints as Organizational Governance

Kim [2026a] frames harness engineering organizationally. Working at Toss, one of South Korea’s largest fintech companies, the author observed that “even with the same model and IDE, the variance in output is extreme.” The proposed solution—a plugin marketplace for distributing lint rules, Git strategies, and testing policies as installable packages—embeds both Context (team-shared conventions) and Constraint (composable enforcement rules) into distributable units. The composability requirement implies path-independent convergence: team leads can install constraint packages in any order without worrying about interaction effects.

This organizational framing adds a dimension absent from OpenAI’s account: **harness engineering as a team coordination mechanism**, where the constraint set becomes a shared contract across engineers with heterogeneous skill levels.

4.1.3 Shoemaker: Layered Verification as Variance Reduction

Shoemaker [2026] articulates the most explicit variance reduction strategy. His development workflow comprises 35+ skills (Enforcement), structured agent instructions (Context), and layered verification including cross-model review (sending diffs to a different AI model). His key insight—“Even when I do [read the code], I am generally trying to figure out what the gap in the harness is so that I don’t have to address this problem again”—reveals the Convergence logic: each gap identified and closed is a permanent reduction in the space of permissible structural variance.

Shoemaker’s approach maps cleanly to the guidance–enforcement spectrum (Table 1): specs operate as Guidance, skills as Enforcement, and cross-model review provides diversity-based error detection outside the spectrum entirely.

4.1.4 Hashimoto: From Infrastructure Idempotency to Code Governance

Hashimoto [2026], founder of HashiCorp and creator of Terraform, independently arrived at the term “harness engineering” in his account of adopting AI coding agents. His six-step adoption journey culminates in Step 5, “Engineer the Harness”: “anytime you find an agent makes a mistake, you take the time to engineer a solution such that the agent never makes that mistake again.” Hashimoto describes two complementary mechanisms: implicit prompting through AGENTS.md files (probabilistic) and “actual, programmed tools” such as scripts and filtered test runners (deterministic)—a dual-layer structure that maps directly to our enforcement spectrum.

The significance of this convergence extends beyond vocabulary. Hashimoto designed Terraform’s idempotent state convergence—the principle that infrastructure configuration, regardless of how many times it is applied, converges to a single declared state [Hummer et al., 2013]. Two decades later, he independently re-discovered an analogous principle at the code-structure level: harness rules, regardless of when they are introduced, should converge code to a canonical structural form. This cross-level re-discovery by the same engineer provides evidence that the convergence principle is not domain-specific but reflects a general pattern in governance of generated artifacts.

Furthermore, Hashimoto’s six-step journey—from chatbot (Step 1) through reproducing manual work (Step 2), end-of-day agents (Step 3), outsourcing “slam dunks” (Step 4), to engineering the harness (Step 5) and continuous agent operation (Step 6)—demonstrates Convergence at the individual practitioner level. Each step incrementally reduces the space of uncontrolled structural decisions, embodying the iterative refinement logic.

4.1.5 Zakas: Tooling as Embedded Governance

Zakas [2026], creator of ESLint, argues that the future of software engineering is “not writing code but orchestrating AI agents, with skills shifting from syntax and implementation to architecture, specification, and feedback loop design.” This is significant precisely because Zakas designed the most widely-used JavaScript linting framework: the toolmaker recognizes that his tool’s role has fundamentally shifted from style enforcement to architectural governance.

4.1.6 Böckeler/Thoughtworks: External Validation and Organizational Implications

Böckeler [2026], a Distinguished Engineer at Thoughtworks, published an analysis of harness engineering on Martin Fowler’s blog, providing external validation from a major software consultancy. Rather than describing an independent discovery, Böckeler synthesizes the emerging discourse and extends it with organizational implications. She categorizes harness components into three groups: (1) context engineering (knowledge bases and dynamic context), (2) architectural constraints (deterministic linters and structural tests), and (3) “garbage collection” (periodic cleanup agents fighting entropy).

Two of her observations connect directly to our framework. First, “the runtime has to be constrained for more AI autonomy”—a formulation that captures the Constraint logic: trust in AI-generated code increases only when the solution space is deliberately narrowed through enforcement. Second, she anticipates “a convergence on a limited number of tech stacks and topologies,” suggesting that Convergence may operate not only within codebases but across organizations adopting similar harness-compatible architectures. Her speculation that harnesses may become “the future service templates”—organizational golden paths for AI-driven software engineering—points to a dimension our framework does not yet address: harness engineering as an organizational coordination mechanism beyond the single-team level.

4.2 Cross-Case Synthesis

The six independent accounts converge on patterns that map directly to the Context–Constraint–Convergence framework:

Table 3: Dimension recurrence across independent harness engineering practices.

Source	Context	Constraint	Convergence
OpenAI	Curated docs/ knowledge base	Custom linters + golden principles	Recurring cleanup agents
Toss	Team-shared conventions	Composable plugin marketplace	Team-level rollout
Shoemaker	Structured specs	35+ skills + layered verification	Gap-finding iteration
Hashimoto	AGENTS.md knowledge	Programmed tools + scripts	6-step adoption journey
Zakas	Architecture documentation	Lint as governance gate	Tool evolution over decades
Böckeler/TW	Context engineering	Constrained runtime	Periodic “garbage collection”

The convergence is notable. While the sources are not fully independent—some reference the term “harness engineering” from OpenAI—their described practices were developed in distinct organizational contexts without shared methodology. Hashimoto arrived at the identical term independently; Toss, Shoemaker, and Zakas described similar practices under different names. Böckeler’s analysis, while building on the OpenAI post, provides external validation from a major consultancy and extends the discourse toward organizational implications. Each source discovered the same needs—knowledge that informs, constraints that govern, and iterative refinement that converges. This convergent emergence suggests that the three dimensions are not artifacts of our framework but intrinsic features of the underlying engineering challenge.

The naming gap. The six sources describe the same phenomenon but share no common analytical vocabulary. OpenAI and Hashimoto independently call it “harness engineering.” Shoemaker calls it “harness-first verification.” Toss frames it as “organizational harness.” Zakas calls it “orchestration.” Böckeler frames it through the lens of “service templates.” The absence of shared vocabulary inhibits cross-pollination and cumulative knowledge building—precisely the gap this paper aims to fill.

5 Discussion

5.1 The Guidance–Enforcement Spectrum

Current practice reveals a natural dual-layer governance model:

Layer	Medium	Mechanism	Strength
Guidance	AGENTS.md, prompts	LLM-dependent	Broad coverage
Enforcement	Linters, type system, CI	Binary pass/fail	Structural guarantees

Neither layer is sufficient alone. Guidance tells the AI *why* the architecture exists and provides examples—essential for generating code that is directionally correct. Enforcement *guarantees* that the output conforms to structural constraints—essential for convergence.

As the convergent evidence in Section 4 demonstrates, this guidance–enforcement structure emerges independently across practitioners: Shoemaker’s specs (Guidance) plus layered verification (Enforcement), OpenAI’s AGENTS.md (Guidance) plus custom linters (Enforcement), and Hashimoto’s AGENTS.md (Guidance) plus “programmed tools” (Enforcement). That the creator of Terraform—a tool whose central guarantee is idempotent state convergence—independently arrived at the same structure for code governance suggests that the principle may generalize across abstraction levels: from infrastructure state to code structure.

5.2 Implications for Software Engineering Governance

The existing discourse frames harness engineering as an operational practice: *how to make AI agents productive*. We argue its deeper significance is an emerging shift in software governance:

From: “How do we ensure the right *people* write code?”

To: “How do we ensure *any agent’s* code conforms to *our architecture*?”

This reframing is significant because it:

1. **Removes agent-specificity.** The same harness constrains humans and AIs identically. A naming rule doesn’t care whether the violating code was typed manually or generated by GPT-5.
2. **Provides deterministic guarantees.** Unlike prompt engineering, lint rules either pass or fail.
3. **Creates verifiable contracts.** CI pipelines with lint gates provide binary, auditable conformance checks.

5.3 The Rigidity–Flexibility Tradeoff

Every constraint trades flexibility for consistency. In practice, restrictive naming rules often require explicit exceptions—cases where the constraint’s granularity proves insufficient for edge cases. This suggests an inherent tension: **the more comprehensive the harness, the more exceptions it requires**, creating a maintenance cost proportional to architectural complexity.

OpenAI navigates this by “enforcing boundaries centrally, allowing autonomy locally” [Lopopolo, 2026]—a principle that maps to the guidance–enforcement spectrum. Strict enforcement applies to cross-cutting concerns (naming, directory structure), while lighter guidance applies to domain-specific patterns where flexibility is needed.

5.4 Limitations

1. **Observational evidence.** Our convergent emergence argument draws on blog posts and practitioner accounts, not controlled experiments. The industry sources may share unacknowledged influences.
2. **Limited empirical validation.** The framework is illustrated through practitioner accounts but has not been validated through controlled case studies or multi-project comparisons.
3. **Ecosystem generality.** The practitioner accounts and examples in this paper are drawn from ecosystems with mature linting and plugin infrastructure. Whether the framework applies with equal explanatory power to ecosystems with different enforcement traditions—such as those centered on compiler-level guarantees or type-system-driven governance—remains an open question.
4. **Maintenance cost unquantified.** We discuss constraint development but not ongoing friction from constraint exceptions and updates.

5. **Semantic gap.** Structural convergence does not guarantee behavioral equivalence—two structurally identical implementations may contain different bugs.

6 Conclusion

We have provided an initial structured articulation of harness engineering, a practice that has rapidly emerged in industry but lacks shared analytical vocabulary. By organizing harness engineering along three dimensions—Context, Constraint, and Convergence—and adopting structural idempotence as the convergence criterion, we offer a framework for reasoning about architectural governance in AI-driven software engineering.

The convergent emergence of similar practices across independent practitioners—from OpenAI’s million-line code-base to individual engineers’ personal workflows—suggests that the underlying phenomenon is substantive and that shared vocabulary is needed. The Context–Constraint–Convergence framework provides this vocabulary: Context captures the knowledge that informs agents, Constraint captures the rules that govern their output, and Convergence captures the iterative process by which the governance system itself evolves.

Our central argument is that the industry discourse—while directionally correct—understates the significance of harness engineering. It is not merely an operational practice for managing AI agents. It may signal an emerging shift in how software quality is governed: from controlling the *identity* of code authors to controlling the *structure* of code artifacts. Well-designed harnesses make parts of this shift verifiable.

The harness does not replace human judgment. It frees human judgment to focus on semantics, design, and intent—the dimensions where variance is not merely tolerable but desirable.

References

- Rocco Lopopolo. Harness engineering: Leveraging codex in an agent-first world. OpenAI Blog, February 2026. URL <https://openai.com/index/harness-engineering/>. Accessed: 2026-03-08.
- Ben Shoemaker. In defense of not reading the code. Personal Blog, February 2026. URL <https://www.benshoemaker.us/writing/in-defense-of-not-reading-the-code/>. Accessed: 2026-03-08.
- Yongsung Kim. Raising the floor of team productivity through harness in the software 3.0 era. Toss Tech Blog, 2026a. URL <https://toss.tech/article/harness-for-team-productivity>. Accessed: 2026-03-08.
- David Heinemeier Hansson. *Agile Web Development with Rails*. Pragmatic Bookshelf, 2004.
- Mitchell Hashimoto. My AI adoption journey. Personal Blog, February 2026. URL <https://mitchellh.com/writing/my-ai-adoption-journey>. Accessed: 2026-03-22.
- Waldemar Hummer, Florian Rosenberg, Fábio Oliveira, and Tamar Eilam. Testing idempotence for infrastructure as code. In *Proc. 14th International Middleware Conference (Middleware)*, pages 368–388, 2013. doi:10.1007/978-3-642-45065-5_19.
- Kief Morris. *Infrastructure as Code*. O’Reilly Media, 2016.
- Stephen C. Johnson. Lint, a C program checker. Technical Report Computer Science Technical Report 65, Bell Laboratories, 1978.
- Moritz Beller, Radjino Bholanath, Shane McIntosh, and Andy Zaidman. Analyzing the state of static analysis: A large-scale evaluation in open source software. In *Proc. IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, pages 470–481, 2016. doi:10.1109/saner.2016.105.
- Andrej Karpathy. Vibe coding. Twitter/X, February 2025.
- Nicholas C. Zakas. From coder to orchestrator: The future of software engineering. Human Who Codes Blog, January 2026. URL <https://humanwhocodes.com/blog/2026/01/coder-orchestrator-future-software-engineering/>. Accessed: 2026-03-08.
- Dewayne E. Perry and Alexander L. Wolf. Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4):40–52, 1992. doi:10.1145/141874.141884.
- Lakshitha de Silva and Dharini Balasubramaniam. Controlling software architecture erosion: A survey. *Journal of Systems and Software*, 85(1):132–151, 2012. doi:10.1016/j.jss.2011.07.036.
- Carliss Y. Baldwin and Kim B. Clark. *Design Rules: The Power of Modularity*. MIT Press, 2000. doi:10.7551/mitpress/2366.001.0001.

- Ward Cunningham. The WyCash portfolio management system. In *Addendum to the Proc. OOPSLA '92*, pages 29–30, 1992. doi:10.1145/157709.157715.
- Peter Gafert. ArchUnit: Unit tests for architecture. In *Proc. Software Architecture Conference*, 2019.
- Jens Knodel and Daniel Popescu. A comparison of static architecture compliance checking approaches. In *Proc. Working IEEE/IFIP Conference on Software Architecture (WICSA)*, 2007. doi:10.1109/wicsa.2007.1.
- Miltiadis Allamanis, Earl T. Barr, Christian Bird, and Charles Sutton. Learning natural coding conventions. In *Proc. 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 281–293, 2014. doi:10.1145/2635868.2635883.
- Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- Alonzo Church and J. Barkley Rosser. Some properties of conversion. *Transactions of the American Mathematical Society*, 39(3):472–482, 1936.
- Jiun Kim. Structural idempotence. March 2026b. doi:10.5281/zenodo.19107646. Preprint.
- Birgitta Böckeler. Harness engineering. Martin Fowler Blog, Thoughtworks, February 2026. URL <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>. Accessed: 2026-03-22.